

Name: Charmi Gaddale**Roll Number:** 23071A0516**AIM:** Implement Naive Bayes using Gaussian and Multinomial NB

```
#Import important libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report
```

Dataset: Loan Approval Dataset

```
import kagglehub
path = kagglehub.dataset_download("vikramamin/bank-loan-approval-lr-dt-rf-and-auc")
```

Using Colab cache for faster access to the 'bank-loan-approval-lr-dt-rf-and-auc' dataset.

```
import os
```

```
print(os.listdir(path))
```

```
['bankloan.csv']
```

```
df = pd.read_csv(os.path.join(path, 'bankloan.csv'))
```

```
df.head()
```

	ID	Age	Experience	Income	ZIP.Code	Family	CCAvg	Education	Mortgage	Personal.Loan	Securities.Account	CD.Account
0	1	25	1	49	91107	4	1.6	1	0	0	1	0
1	2	45	19	34	90089	3	1.5	1	0	0	1	0
2	3	39	15	11	94720	1	1.0	1	0	0	0	0
3	4	35	9	100	94112	1	2.7	2	0	0	0	0
4	5	35	8	45	91330	4	1.0	2	0	0	0	0

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
df = df.drop(columns=["ID"])
```

```
print(df.columns)
```

```
Index(['Age', 'Experience', 'Income', 'ZIP.Code', 'Family', 'CCAvg',
      'Education', 'Mortgage', 'Personal.Loan', 'Securities.Account',
      'CD.Account', 'Online', 'CreditCard'],
      dtype='object')
```

```
# initialize encoder
encoder = LabelEncoder()

# list of categorical columns to encode
categorical_columns = [
    'Education',
    'Personal.Loan', # target before rename
    'Securities.Account',
    'CD.Account',
    'Online',
    'CreditCard'
]
```

```
# apply label encoding
for col in categorical_columns:
    df[col] = encoder.fit_transform(df[col])

# rename target column
df.rename(columns={"Personal Loan": "Approved"}, inplace=True)

# show updated dataframe info
print(df.info())
print(df.head())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Age                    5000 non-null   int64
1   Experience              5000 non-null   int64
2   Income                  5000 non-null   int64
3   ZIP.Code                5000 non-null   int64
4   Family                  5000 non-null   int64
5   CCAvg                   5000 non-null   float64
6   Education               5000 non-null   int64
7   Mortgage                5000 non-null   int64
8   Personal.Loan           5000 non-null   int64
9   Securities.Account       5000 non-null   int64
10  CD.Account              5000 non-null   int64
11  Online                  5000 non-null   int64
12  CreditCard              5000 non-null   int64
dtypes: float64(1), int64(12)
memory usage: 507.9 KB
None
```

	Age	Experience	Income	ZIP.Code	Family	CCAvg	Education	Mortgage	\
0	25	1	49	91107	4	1.6	0	0	
1	45	19	34	90089	3	1.5	0	0	
2	39	15	11	94720	1	1.0	0	0	
3	35	9	100	94112	1	2.7	1	0	
4	35	8	45	91330	4	1.0	1	0	

	Personal.Loan	Securities.Account	CD.Account	Online	CreditCard
0	0	1	0	0	0
1	0	1	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	1

```
# Step 2: Define Features and Target
X = df.drop(columns=["Personal.Loan"]) #Features
y = df["Personal.Loan"]               # Target

# Step 3: Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 4: Train the Naive Bayes model
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)

# Step 5: Make predictions
y_pred = nb_model.predict(X_test)

# Step 6: Evaluate the model
print("Accuracy:", round(accuracy_score(y_test, y_pred), 2))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

Accuracy: 0.9

Classification Report:

	precision	recall	f1-score	support
0	0.95	0.93	0.94	895
1	0.52	0.62	0.57	105

accuracy			0.90	1000
macro avg	0.74	0.78	0.76	1000
weighted avg	0.91	0.90	0.90	1000

```
print(X.columns)
```

```
Index(['Age', 'Experience', 'Income', 'ZIP.Code', 'Family', 'CCAvg',
      'Education', 'Mortgage', 'Securities.Account', 'CD.Account', 'Online',
      'CreditCard'],
      dtype='object')
```

```
new_customer = np.array([[ 35, 8, 96, 91330, 4, 1.0, 2, 0, 0, 0, 0, 1]])
```

```
prediction = nb_model.predict(new_customer)
```

```
if prediction[0] == 1:
    print("Loan Approved")
else:
    print("Loan Not Approved")
```

```
ved
/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but GaussianNB
n(
```

```
prediction = nb_model.predict(new_customer)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names,
warnings.warn(
```

```
prob = nb_model.predict_proba(new_customer)
```

```
print("Probability [Not Approved, Approved]:", prob)
```

```
Probability [Not Approved, Approved]: [[0.99316878 0.00683122]]
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names,
warnings.warn(
```

Inference: Gaussian Naive Bayes is used when the input features are continuous numerical values that follow a normal distribution.

In the bank loan dataset:

- Age
- Income
- Experience
- Mortgage are already numeric.

So the model directly works on these numbers and assumes they follow a **Gaussian distribution**.

No extra transformation like **vectorization** is required because the data is already in **numerical format**.

Multinomial Naive Bayes Classifier for Spam Detection

```
import pandas as pd
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report
```

```
import kagglehub
path = kagglehub.dataset_download("abdallahwagih/spam-emails")
```

```
Using Colab cache for faster access to the 'spam-emails' dataset.
```

```
print(os.listdir(path))
```

```
['spam.csv']
```

```
df=pd.read_csv(os.path.join(path, 'spam.csv'))
```

```
df.info()
df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Category    5572 non-null   object
1   Message     5572 non-null   object
dtypes: object(2)
memory usage: 87.2+ KB
```

	Category	Message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...



Next steps:

[Generate code with df](#)
[New interactive sheet](#)

```
print(df.columns)
```

```
Index(['Category', 'Message'], dtype='object')
```

```
df = df.iloc[:, :2] # keep only first two columns
df.columns = ["label", "message"]
```

```
print(df.columns)
```

```
Index(['label', 'message'], dtype='object')
```

```
df["label"] = df["label"].map({"spam": 1, "ham": 0})
df.dropna(inplace=True)
```

```
#Data Preprocessing(Removing the white spaces + Converting the words to base form)
nltk.download("stopwords")
stop_words = set(stopwords.words("english"))
stemmer = PorterStemmer()
```

```
def preprocess(text):
    text = text.lower()
    text = re.sub(r"\W", " ", text)
    text = " ".join([stemmer.stem(word)
                     for word in text.split()
                     if word not in stop_words])
    return text
```

```
df["clean_message"] = df["message"].apply(preprocess)
```

```
df.head()
```

1 to 5 of 5 entries

Filter



index	label	message	clean_message
0	0	Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...	go jurong point crazi avail bugi n great world la e buffet cine got amor wat
1	0	Ok lar... Joking wif u oni...	ok lar joke wif u oni
2	1	Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive entry question(std txt rate)T&C's apply 08452810075over18's	free entri 2 wkly comp win fa cup final tkt 21st may 2005 text fa 87121 receiv entri question std txt rate c appli 08452810075over18
3	0	U dun say so early hor... U c already then say...	u dun say earli hor u c already say
4	0	Nah I don't think he goes to usf, he lives around here though	nah think goe usf live around though

Show 25 per page

Like what you see? Visit the [data table notebook](#) to learn more about interactive tables

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
#Train Test Split
X_train, X_test, y_train, y_test = train_test_split(
    df["clean_message"], df["label"],
    test_size=0.2, random_state=42
)
```

```
#Convert Text to TF-IDF
vectorizer = TfidfVectorizer()

X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)
```

```
#Train Multinomial Naive Bayes
model = MultinomialNB()
model.fit(X_train_tfidf, y_train)
```

▼ MultinomialNB ⓘ ?

MultinomialNB()

```
#Evaluate
y_pred = model.predict(X_test_tfidf)

print("Accuracy:", round(accuracy_score(y_test, y_pred), 2))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Accuracy: 0.97

Classification Report:

	precision	recall	f1-score	support
0	0.97	1.00	0.98	966
1	1.00	0.77	0.87	149
accuracy			0.97	1115
macro avg	0.98	0.89	0.93	1115
weighted avg	0.97	0.97	0.97	1115

Predict a New Email Message

```
new_msg = ["Congratulations! You have won a free prize. Claim now!"]
```

```
new_msg_clean = [preprocess(new_msg[0])]
```

```
print(new_msg_clean)
```

```
['congratul free prize claim']
```

```
new_msg_tfidf = vectorizer.transform(new_msg_clean)
```

```
print(new_msg_tfidf)
```

```
<Compressed Sparse Row sparse matrix of dtype 'float64'
  with 4 stored elements and shape (1, 6428)>
Coords      Values
(0, 1633)    0.45496080328024946
(0, 1744)    0.6425254281249089
(0, 2550)    0.38021548791909465
(0, 4561)    0.4853946069312448
```

```
prediction = model.predict(new_msg_tfidf)
```

```
if prediction[0] == 1:
    print("Spam Email")
else:
    print("Not Spam Email")
```

```
Spam Email
```

```
prob = model.predict_proba(new_msg_tfidf)
print("Probability [Not Spam, Spam]:", prob)
```

```
Probability [Not Spam, Spam]: [[0.04889263 0.95110737]]
```

Inference: Multinomial Naive Bayes is mainly used for text classification problems, such as spam detection.

But machine learning models cannot understand raw text like:

"Win a free prize now"

Models only work with numbers.

So we convert text into numerical form using **TF-IDF vectorization**.